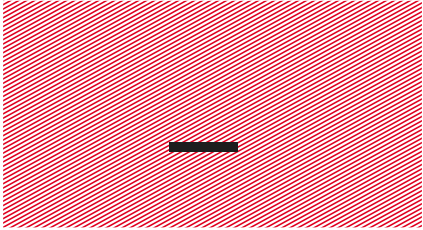


# — IoT Devices

*Les 8 – RTOS deel 2*





# Binary Semaphore





## Binary Semaphore

- **Semafoor** = een datatype dat gebruikt kan worden om de **acties** van meerdere threads te synchroniseren.
- De meest eenvoudige is de '**binaire semafoor**'. Hierbij kan een thread de semafoor 'opeisen' (**acquire**) en kan een andere thread de semafoor 'vrijgeven' (**release**).
- Is de binaire semafoor **reeds opgeëist**, dan worden alle threads die hem willen (acquire) in '**blocked** state' gezet, tot een thread de semafoor vrijgeeft (release).
- Dit concept kan je gebruiken om **de ene thread op de andere te laten wachten**... Opnieuw een soort **synchronisatie** dus!
- Wordt eigenlijk gemaakt met een queue met één element...

# Binary Semaphore

STM32CubeMX ondersteunt het maken van 'semaphores'...

The screenshot shows the STM32CubeMX Pinout & Configuration window. The left sidebar lists various middleware and software packages, with FREERTOS selected. The main area shows the configuration for the selected package. The 'Timers and Semaphores' option is highlighted in the 'Configuration' section. Below it, the 'Binary Semaphores' table is visible, showing a semaphore named 'myBinarySemaphore' with a 'Dynamic' allocation and a 'NULL' control block name. The 'Counting Semaphores' section is also visible below.

| Timer Name          | Callback | Type       | Code Generation Option | Parameter          | Allocation         | Control Block Name |
|---------------------|----------|------------|------------------------|--------------------|--------------------|--------------------|
| Binary Semaphores   |          |            |                        |                    |                    |                    |
| Semaphore Name      |          | Allocation |                        | Control Block Name |                    |                    |
| myBinarySemaphore   |          | Dynamic    |                        | NULL               |                    |                    |
| Counting Semaphores |          |            |                        |                    |                    |                    |
| Semaphore Name      |          | Count      | Allocation             |                    | Control Block Name |                    |

# Binary Semaphore

Het aanmaken van een **nieuwe semafoor** kan in C-taal met de functie `osSemaphoreNew()`.

```
osSemaphoreId_t osSemaphoreNew ( uint32_t      max_count,  
                                uint32_t      initial_count,  
                                const osSemaphoreAttr_t * attr  
                                )
```

## Parameters

- [in] **max\_count** maximum number of available tokens.
- [in] **initial\_count** initial number of available tokens.
- [in] **attr** semaphore attributes; NULL: default values.

## Returns

semaphore ID for reference by other functions or NULL in case of error.

The function **osSemaphoreNew** creates and initializes a semaphore object that is used to manage access to shared resources and returns the pointer to the semaphore object identifier or **NULL** in case of an error. It can be safely called before the RTOS is started (call to **osKernelStart**), but not before it is initialized (call to **osKernelInitialize**).

The parameter *max\_count* specifies the maximum number of available tokens. A *max\_count* value of 1 creates a binary semaphore.

The parameter *initial\_count* sets the initial number of available tokens.

The parameter *attr* specifies additional semaphore attributes. Default attributes will be used if set to **NULL**.

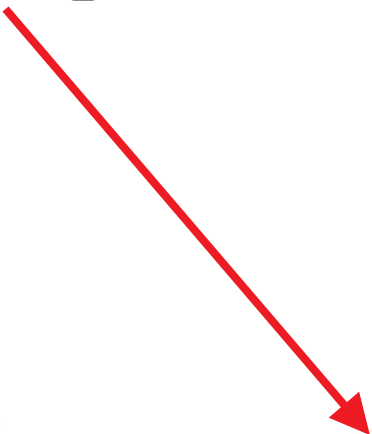
Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_SemaphoreMgmt.html#ga2a39806ace781a0008a4374ca701b14a](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_SemaphoreMgmt.html#ga2a39806ace781a0008a4374ca701b14a)

## Binary Semaphore

Het aanmaken van een **nieuwe semafoor** kan in C-taal met de functie `osSemaphoreNew()`.

```
73 /* Definitions for myBinarySemaphore */
74 osSemaphoreId_t myBinarySemaphoreHandle;
75 const osSemaphoreAttr_t myBinarySemaphore_attributes = {
76     .name = "myBinarySemaphore"
77 };
```

```
142 /* Create the semaphores(s) */
143 /* creation of myBinarySemaphore */
144 myBinarySemaphoreHandle = osSemaphoreNew(1, 1, &myBinarySemaphore_attributes);
```



# Binary Semaphore

```
52 /* Definitions for SemaAcqThread */
53 osThreadId_t SemaAcqThreadHandle;
54 □ const osThreadAttr_t SemaAcqThread_attributes = {
55     .name = "SemaAcqThread",
56     .stack_size = 128 * 4,
57     .priority = (osPriority_t) osPriorityNormal,
58 };
59 /* Definitions for SemaRelThread1 */
60 osThreadId_t SemaRelThread1Handle;
61 □ const osThreadAttr_t SemaRelThread1_attributes = {
62     .name = "SemaRelThread1",
63     .stack_size = 128 * 4,
64     .priority = (osPriority_t) osPriorityNormal,
65 };
66 /* Definitions for SemaRelThread2 */
67 osThreadId_t SemaRelThread2Handle;
68 □ const osThreadAttr_t SemaRelThread2_attributes = {
69     .name = "SemaRelThread2",
70     .stack_size = 128 * 4,
71     .priority = (osPriority_t) osPriorityNormal,
72 };
```

# Binary Semaphore

```
159  /* creation of SemaAcqThread */
160  SemaAcqThreadHandle = osThreadNew(SemaphoreAcquire, NULL, &SemaAcqThread_attributes);
161
162  /* creation of SemaRelThread1 */
163  SemaRelThread1Handle = osThreadNew(SemaphoreReleaseByDelay, NULL, &SemaRelThread1_attributes);
164
165  /* creation of SemaRelThread2 */
166  SemaRelThread2Handle = osThreadNew(SemaphoreReleaseByButton, NULL, &SemaRelThread2_attributes);
```



# Binary Semaphore

```
474 /* USER CODE END Header_SemaphoreReleaseByButton */
475 void SemaphoreReleaseByButton(void *argument)
476 {
477     /* USER CODE BEGIN SemaphoreReleaseByButton */
478     bool swlHelp = false;
479
480     /* Infinite loop */
481     for(;;)
482     {
483         // Flankdetectie doen op SW1 (met antidenderwachtijd).
484         if(SW1Active() && (swlHelp == false))
485         {
486             swlHelp = true;
487
488             // De binaire semafoor beschikbaar stellen voor gebruik in een andere thread.
489             // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_SemaphoreMgmt.html.
490             osSemaphoreRelease(myBinarySemaphoreHandle);
491         }
492
493         if(!SW1Active())
494             swlHelp = false;
495
496         // Antidenderwachtijd.
497         osDelay(10);
498     }
499     /* USER CODE END SemaphoreReleaseByButton */
500 }
```



# Binary Semaphore

```
452 void SemaphoreReleaseByDelay(void *argument)
453 {
454     /* USER CODE BEGIN SemaphoreReleaseByDelay */
455     /* Infinite loop */
456     for(;;)
457     {
458         // De binaire semafoor beschikbaar stellen voor gebruik in een andere thread.
459         // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_SemaphoreMgmt.html.
460         osSemaphoreRelease(myBinarySemaphoreHandle);
461
462         // Even wachten.
463         osDelay(3000);
464     }
465     /* USER CODE END SemaphoreReleaseByDelay */
466 }
```



# Binary Semaphore

```
425 void SemaphoreAcquire(void *argument)
426 {
427     /* USER CODE BEGIN 5 */
428     uint8_t ledState = 0;
429
430     /* Infinite loop */
431     for(;;)
432     {
433         // Wachten (in blocked state) tot de binaire semafoor beschikbaar is.
434         // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_SemaphoreMgmt.html.
435         osSemaphoreAcquire(myBinarySemaphoreHandle, osWaitForever);
436
437         // LED-teller maken.
438         ByteToLeds(ledState++);
439
440         // OPM: dit concept kan je in principe ook uitwerken met Event en/of Thread flags...
441     }
442     /* USER CODE END 5 */
443 }
```

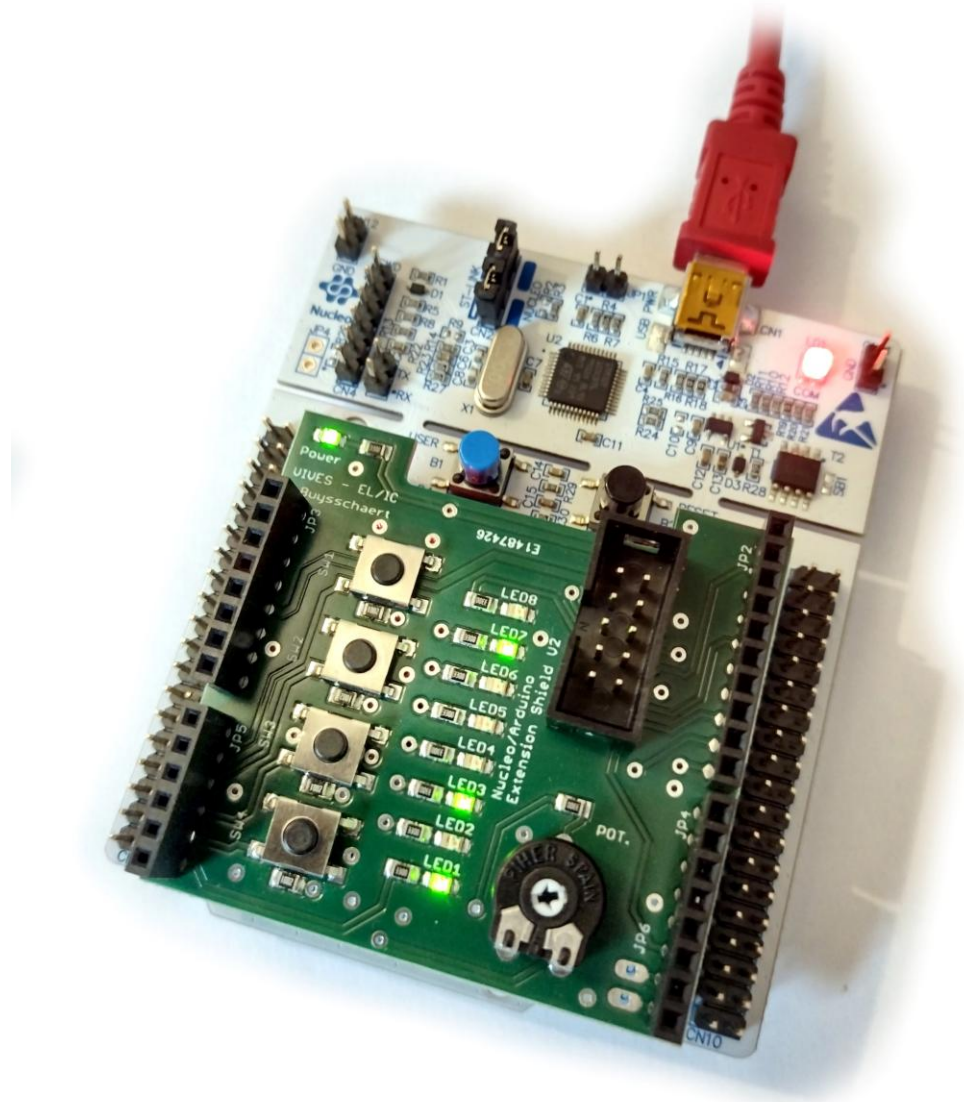
Zie verder..

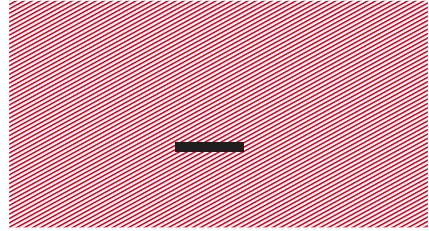


# Binary Semaphore

Test de code...

De LED's moeten binair tellen  
iedere 3 seconden, of  
wanneer je op SW1 drukt...





# Counting Semaphore



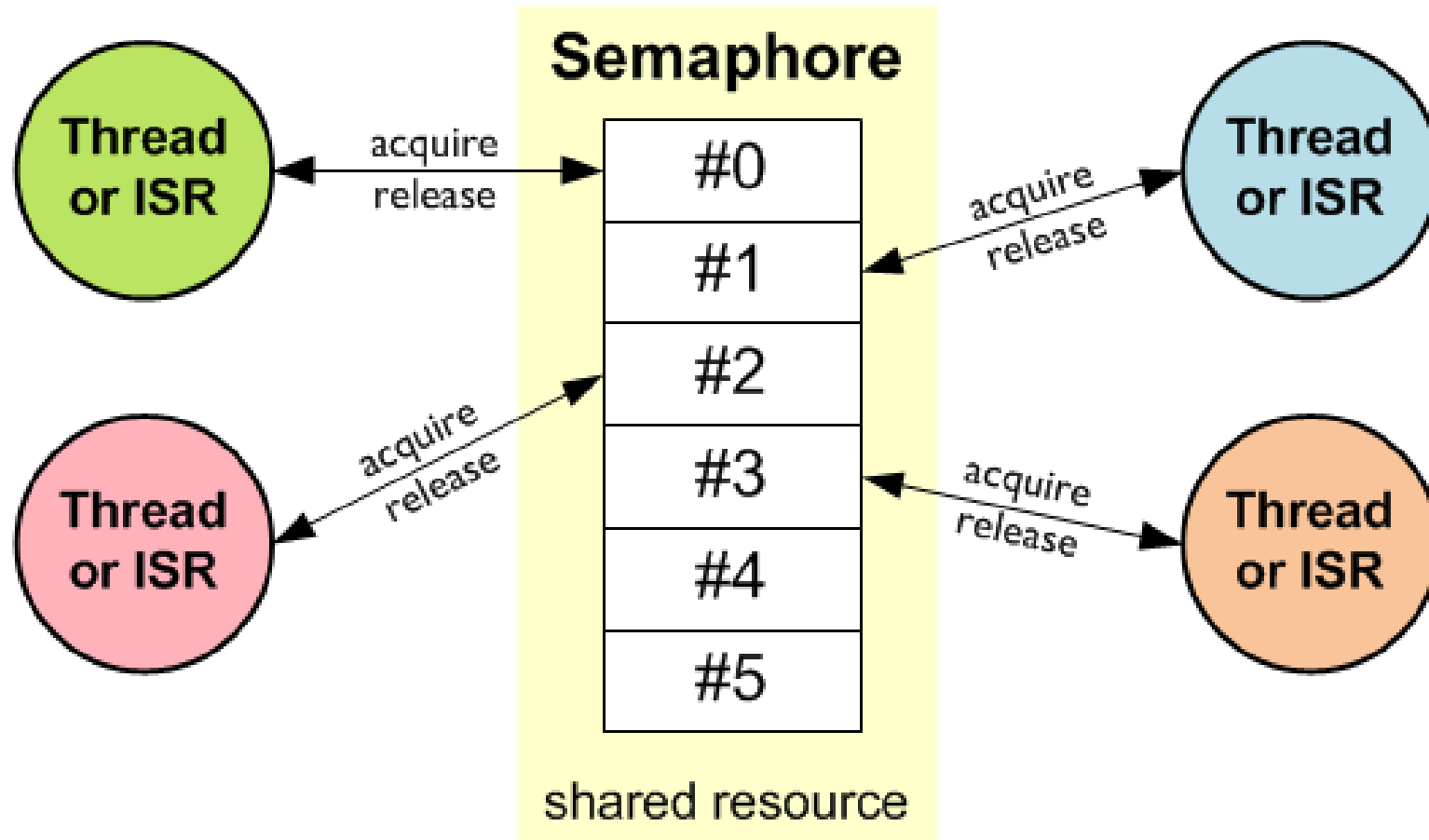




## Counting Semaphore

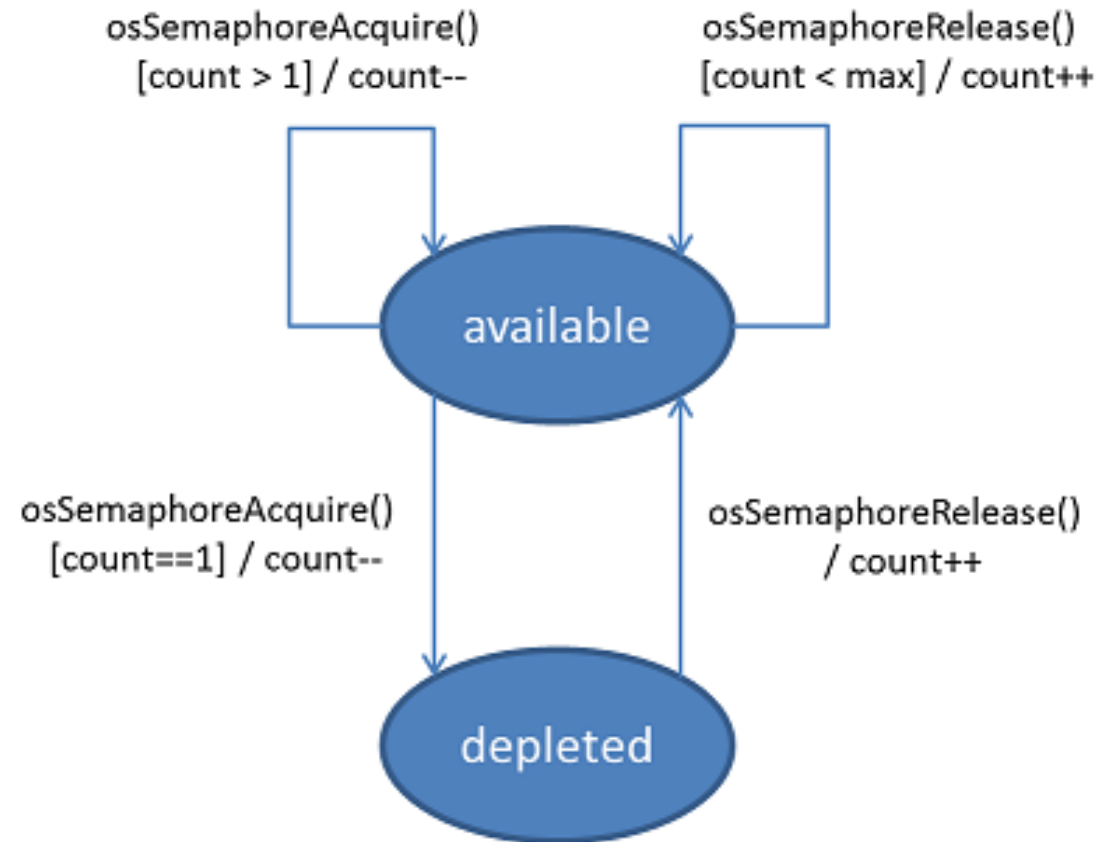
- Een ander type semafoor is de 'tellende semafoor' (**counting semaphore**).
- Ook hierbij kan een thread de semafoor 'opeisen' (**acquire**) en kan een thread de semafoor 'vrijgeven' (**release**).
- Acquire = -1, release = +1...
- Staat de teller/inhoud op nul, dan worden alle threads die hem willen (acquire) in '**blocked** state' gezet, tot minstens één thread de semafoor opnieuw verhoogt (release).
- Dit concept kan je gebruiken om **bijvoorbeeld bij te houden hoeveel bytes er via de UART binnenkomen via een interrupt**... Opnieuw een soort **synchronisatie** dus!
- Wordt eigenlijk gemaakt met een queue met meerdere elementen...

# Counting Semaphore



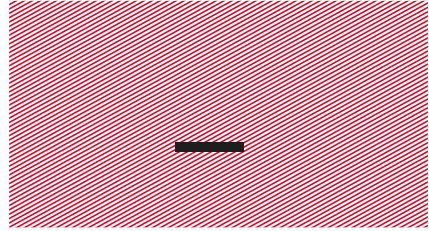
Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_SemaphoreMgmt.html](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_SemaphoreMgmt.html)

# Counting Semaphore



Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_SemaphoreMgmt.html](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_SemaphoreMgmt.html)





# Event Flags



## Event Flags

- **Event flag** = een datatype dat gebruikt kan worden om een **gebeurtenis** van de ene naar de andere thread te melden.
- Zolang de event flag **niet gezet** is (set) door de ene thread, **wacht**(en) de andere thread(s) in blocked state.
- Opnieuw een soort **synchronisatie** dus!
- Ze zijn dikwijls **beter geschikt dan binaire semaforen...** (efficiëntie).
- De vlaggen worden **bewaard** in een 32-bit getal. FreeRTOS ondersteunt er echter maar **24 bit** daarvan.
- Kies zelf **welke bit** voor jou belangrijk is...



## Event Flags

- Standaard wordt gewacht op het **hoog worden** van eender welke vlag (*osFlagsWaitAny*).
- Als je op alle vlaggen wil wachten, moet je dat speciaal instellen.
- Bij het lezen van de vlaggen, worden die ook **automatisch gewist**. Wil je dat niet, geef dat dan aan in de opties.

The *options* parameter specifies the wait condition:

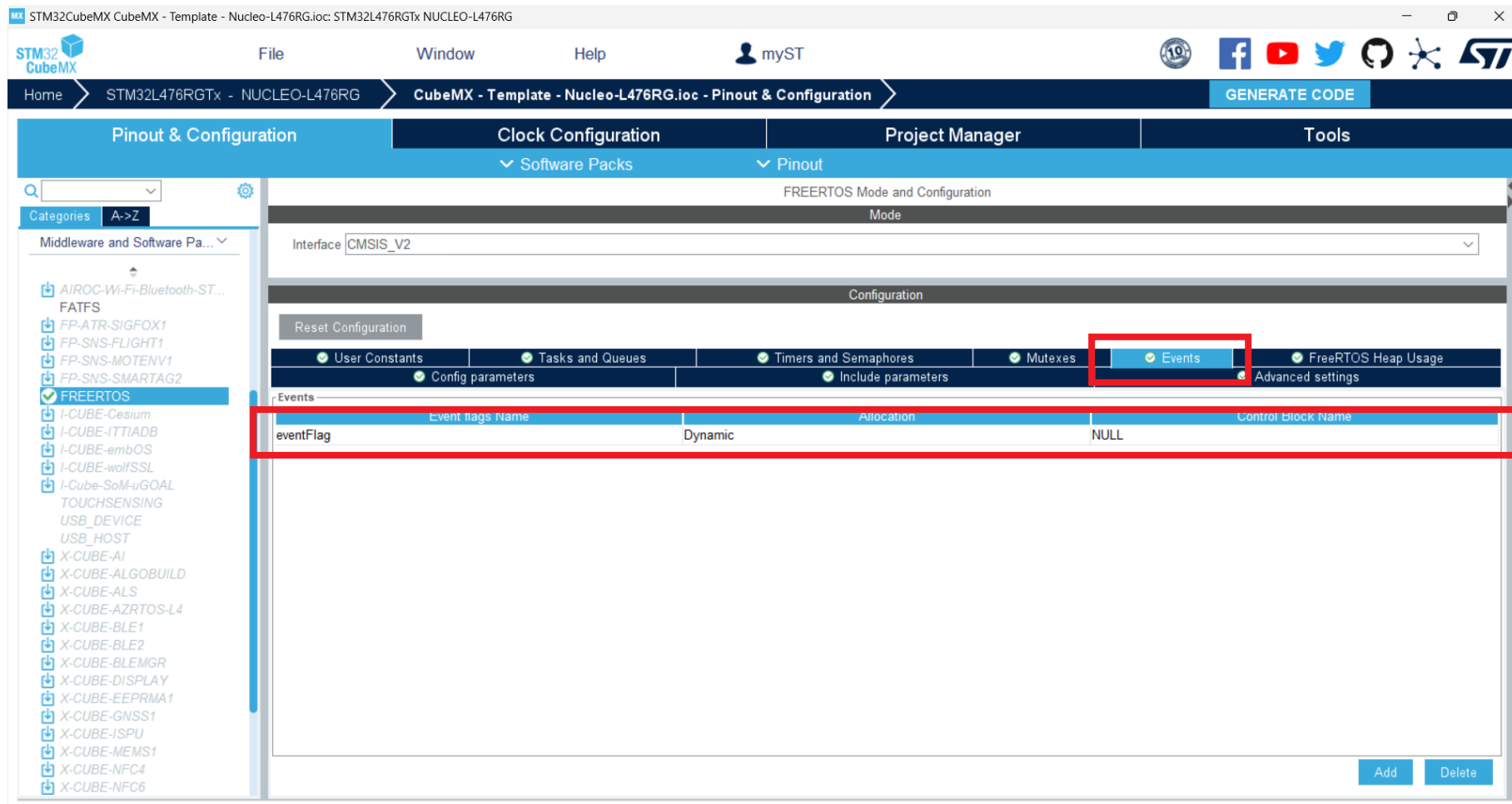
| Option                |                                                           |
|-----------------------|-----------------------------------------------------------|
| <b>osFlagsWaitAny</b> | Wait for any flag (default).                              |
| <b>osFlagsWaitAll</b> | Wait for all flags.                                       |
| <b>osFlagsNoClear</b> | Do not clear flags which have been specified to wait for. |

Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_EventFlags.html](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_EventFlags.html)



# Event Flags

STM32CubeMX ondersteunt het maken van 'events'...



## Event Flags

Het aanmaken van een **nieuw event** kan in C-taal met de functie *osEventFlagsNew()*.

```
osEventFlagsId_t osEventFlagsNew ( const osEventFlagsAttr_t * attr )
```

### Parameters

[in] **attr** event flags attributes; NULL: default values.

### Returns

event flags ID for reference by other functions or NULL in case of error.

The function **osEventFlagsNew** creates a new event flags object that is used to send events across threads and returns the pointer to the event flags object identifier or **NULL** in case of an error. It can be safely called before the RTOS is started (call to **osKernelStart**), but not before it is initialized (call to **osKernelInitialize**).

The parameter *attr* sets the event flags attributes (refer to **osEventFlagsAttr\_t**). Default attributes will be used if set to **NULL**, i.e. kernel memory allocation is used for the event control block.

Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_EventFlags.html#gab14b1caeb12ffa42cce1bfe889cd07df](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_EventFlags.html#gab14b1caeb12ffa42cce1bfe889cd07df)

## Event Flags

Het aanmaken van een **nieuw event** kan in C-taal met de functie *osEventFlagsNew()*.

```
73  /* Definitions for eventFlag */
74  osEventFlagsId_t eventFlagHandle;
75  const osEventFlagsAttr_t eventFlag_attributes = {
76  |  .name = "eventFlag"
77  | };

169  /* creation of eventFlag */
170  eventFlagHandle = osEventFlagsNew(&eventFlag_attributes);
```



# Event Flags

```
52 /* Definitions for EventWaitThread */
53 osThreadId_t EventWaitThreadHandle;
54 □ const osThreadAttr_t EventWaitThread_attributes = {
55     .name = "EventWaitThread",
56     .stack_size = 128 * 4,
57     .priority = (osPriority_t) osPriorityNormal,
58 };
59 /* Definitions for EventSetThread1 */
60 osThreadId_t EventSetThread1Handle;
61 □ const osThreadAttr_t EventSetThread1_attributes = {
62     .name = "EventSetThread1",
63     .stack_size = 128 * 4,
64     .priority = (osPriority_t) osPriorityNormal,
65 };
66 /* Definitions for EventSetThread2 */
67 osThreadId_t EventSetThread2Handle;
68 □ const osThreadAttr_t EventSetThread2_attributes = {
69     .name = "EventSetThread2",
70     .stack_size = 128 * 4,
71     .priority = (osPriority_t) osPriorityNormal,
72 };

155 /* creation of EventWaitThread */
156 EventWaitThreadHandle = osThreadNew(EventWait, NULL, &EventWaitThread_attributes);
157
158 /* creation of EventSetThread1 */
159 EventSetThread1Handle = osThreadNew(EventSetByDelay, NULL, &EventSetThread1_attributes);
160
161 /* creation of EventSetThread2 */
162 EventSetThread2Handle = osThreadNew(EventSetByButton, NULL, &EventSetThread2_attributes);
```



# Event Flags

```
454 void EventSetByDelay(void *argument)
455 {
456     /* USER CODE BEGIN EventSetByDelay */
457     /* Infinite loop */
458     for(;;)
459     {
460         // De Event Flag beschikbaar stellen (setten) voor gebruik in een andere thread.
461         // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_EventFlags.html.
462         osEventFlagsSet(eventFlagHandle, INCREMENT_FLAG);
463
464         // Even wachten.
465         osDelay(3000);
466     }
467     /* USER CODE END EventSetByDelay */
468 }
```



```
99 /* USER CODE BEGIN Private defines */
100 #define INCREMENT_FLAG 0x00000001 // Vlag die aangeeft of een getal mag verhogen.
101 /* USER CODE END Private defines */
```

# Event Flags

```
477 void EventSetByButton(void *argument)
478 {
479     /* USER CODE BEGIN EventSetByButton */
480     bool swlHelp = false;
481
482     /* Infinite loop */
483     for(;;)
484     {
485         // Flankdetectie doen op SW1 (met antidenderwachtijd).
486         if(SW1Active() && (swlHelp == false))
487         {
488             swlHelp = true;
489
490             // De Event Flag beschikbaar stellen (setten) voor gebruik in een andere thread.
491             // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_EventFlags.html.
492             osEventFlagsSet(eventFlagHandle, INCREMENT_FLAG);
493         }
494
495         if(!SW1Active())
496             swlHelp = false;
497
498         // Antidenderwachtijd.
499         osDelay(10);
500     }
```



# Event Flags

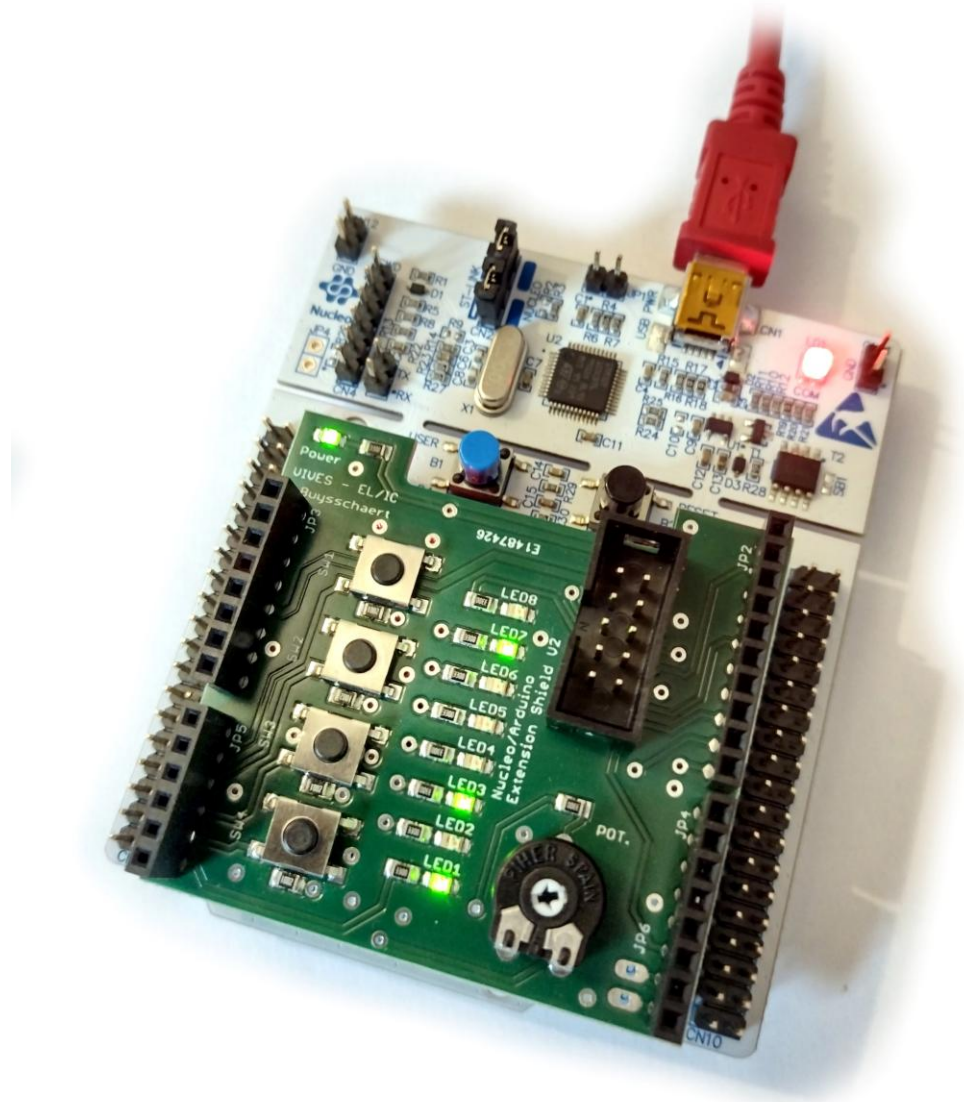
```
425 void EventWait(void *argument)
426 {
427     /* USER CODE BEGIN 5 */
428     uint8_t ledState = 0;
429
430     /* Infinite loop */
431     for(;;)
432     {
433         // Wachten (in blocked state) tot een Event Flag beschikbaar is (als alternatief op het wachten van op een semafoor).
434         // Zie: https://arm-software.github.io/CMSIS\_5/RTOS2/html/group\_\_CMSIS\_\_RTOS\_\_EventFlags.html.
435         // De definitie van INCREMENT_FLAG kan je vinden in main.h.
436         // OPM: bij het vaststellen dat een vlag geset is, wordt automatisch de vlag weer gereset voor volgend gebruik.
437         osEventFlagsWait(eventFlagHandle, INCREMENT_FLAG, osFlagsWaitAll, osWaitForever);
438
439         // LED-teller maken.
440         ByteToLeds(ledState++);
441
442         // OPM: dit concept kan je in principe ook uitwerken met Event en/of Thread flags...
443     }
444     /* USER CODE END 5 */
445 }
```

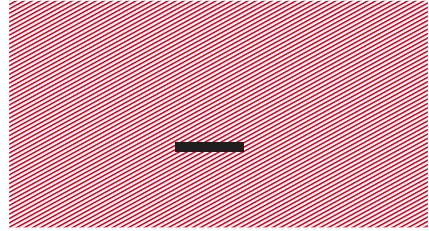


## Event Flags

Test de code...

De LED's moeten binair tellen  
iedere 3 seconden, of  
wanneer je op SW1 drukt...





# Mutex





## Mutex

- **Mutex** = een datatype dat gebruikt kan worden om **gedeelde (hardware)bronnen** tussen threads te 'synchroniseren'.
- Eén tread kan een 'hardware resource' opeisen zodat een andere thread het gebruik van die **resource niet kan verstoren**.
- Mutex = **MUT**ual **EX**clusion.
- Ze verschillen van semaforen omdat ze:
  - meestal opgeëist en vrijgegeven worden door dezelfde thread.
  - **priority inheritance** implementeren (zie verder).



## Mutex

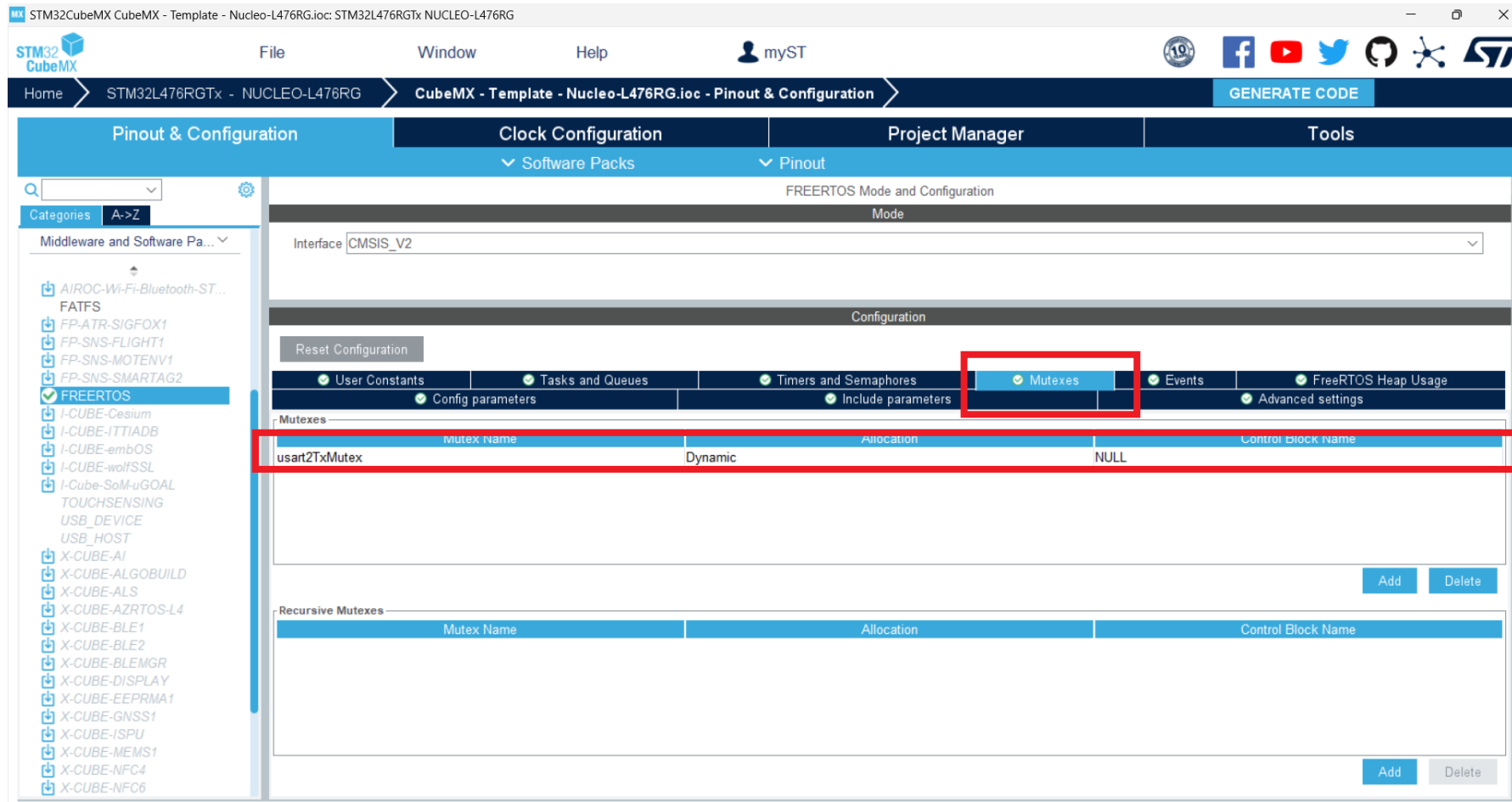
- Typisch **voorbeeld** -> meerdere threads willen gebruik maken van **dezelfde**:
  - **UART**
  - **I<sup>2</sup>C**
  - **SPI**
  - **CAN**
  - ...
- Zeker als die hardware in polling mode gebruikt wordt kan je het gebruik van de Mutex goed zien...





# Mutex

STM32CubeMX ondersteunt het maken van 'mutexes'...





# Mutex

Het aanmaken van een **nieuwe mutex** kan in C-taal met de functie *osMutexNew()*.

```
osMutexId_t osMutexNew ( const osMutexAttr_t * attr )
```

## Parameters

[in] **attr** mutex attributes; NULL: default values.

## Returns

mutex ID for reference by other functions or NULL in case of error.

The function **osMutexNew** creates and initializes a new mutex object and returns the pointer to the mutex object identifier or **NULL** in case of an error. It can be safely called before the RTOS is started (call to **osKernelStart**), but not before it is initialized (call to **osKernelInitialize**).

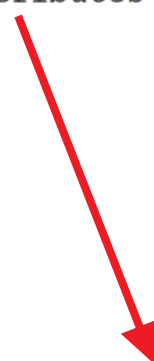
The parameter *attr* sets the mutex object attributes (refer to **osMutexAttr\_t**). Default attributes will be used if set to **NULL**.

Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_\\_CMSIS\\_\\_RTOS\\_\\_MutexMgmt.html#gab90920022ab944296821368ef6bb52f8](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group__CMSIS__RTOS__MutexMgmt.html#gab90920022ab944296821368ef6bb52f8)

Het aanmaken van een **nieuwe mutex** kan in C-taal met de functie *osMutexNew()*.

```
81  /* Definitions for usart2TxMutex */
82  osMutexId_t usart2TxMutexHandle;
83  const osMutexAttr_t usart2TxMutex_attributes = {
84      .name = "usart2TxMutex"
85  };
```

```
147  /* creation of usart2TxMutex */
148  usart2TxMutexHandle = osMutexNew(&usart2TxMutex_attributes);
```



# Mutex

```
53 /* Definitions for BlinkLed1Thread */
54 osThreadId_t BlinkLed1ThreadHandle;
55 □ const osThreadAttr_t BlinkLed1Thread_attributes = {
56     .name = "BlinkLed1Thread",
57     .stack_size = 128 * 4,
58     .priority = (osPriority_t) osPriorityNormal,
59 };
60 /* Definitions for Usart2TxThread */
61 osThreadId_t Usart2TxThreadHandle;
62 □ const osThreadAttr_t Usart2TxThread_attributes = {
63     .name = "Usart2TxThread",
64     .stack_size = 128 * 4,
65     .priority = (osPriority_t) osPriorityNormal,
66 };
67 /* Definitions for Usart2RxThread */
68 osThreadId_t Usart2RxThreadHandle;
69 □ const osThreadAttr_t Usart2RxThread_attributes = {
70     .name = "Usart2RxThread",
71     .stack_size = 128 * 4,
72     .priority = (osPriority_t) osPriorityNormal,
73 };
74 /* Definitions for AdConvThread */
75 osThreadId_t AdConvThreadHandle;
76 □ const osThreadAttr_t AdConvThread_attributes = {
77     .name = "AdConvThread",
78     .stack_size = 128 * 4,
79     .priority = (osPriority_t) osPriorityNormal,
80 };
```



# Mutex

```
167  /* creation of BlinkLed1Thread */
168  BlinkLed1ThreadHandle = osThreadNew(BlinkLed1, NULL, &BlinkLed1Thread_attributes);
169
170  /* creation of Usart2TxThread */
171  Usart2TxThreadHandle = osThreadNew(Usart2Tx, NULL, &Usart2TxThread_attributes);
172
173  /* creation of Usart2RxThread */
174  Usart2RxThreadHandle = osThreadNew(Usart2Rx, NULL, &Usart2RxThread_attributes);
175
176  /* creation of AdConvThread */
177  AdConvThreadHandle = osThreadNew(AdConversion, NULL, &AdConvThread_attributes);
```

# Mutex

```
458 void Usart2Tx(void *argument)
459 {
460     /* USER CODE BEGIN Usart2Tx */
461     /* Infinite loop */
462     for(;;)
463     {
464         // Omdat er meerdere threads zijn die willen schrijven naar USART2, moet je de peripheral
465         // opeisen via een mutex. Deze handeling zorgt ervoor dat je hier wacht, totdat de mutex
466         // beschikbaar is. Daardoor wordt ook gewacht met het versturen van info over de USART2...
467         osMutexAcquire(usart2TxMutexHandle, osWaitForever);
468
469         // Teken van leven geven via UART2.
470         HAL_UART_Transmit(&huart2, (uint8_t*)MESSAGE, strlen(MESSAGE), HAL_MAX_DELAY);
471         // of
472         //printf(MESSAGE);
473
474         // USART2 weer vrijgeven voor gebruik door een andere thread.
475         osMutexRelease(usart2TxMutexHandle);
476
477         // Even wachten.
478         osDelay(1000);
479     }
480     /* USER CODE END Usart2Tx */
481 }
```



# Mutex

```
519 void AdConversion(void *argument)
520 {
521     /* USER CODE BEGIN AdConversion */
522     uint16_t adValue = 0;
523
524     /* Infinite loop */
525     for(;;)
526     {
527         // AD-omzetting starten.
528         adValue = GetAdValue(&hadc1);
529
530         // USART2 opeisen via een mutex.
531         osMutexAcquire(usart2TxMutexHandle, osWaitForever);
532
533         // Waarde op USART2 zetten. Let op: er is nog een ander thread die ook zaken wil versturen op USART2!
534         // Gebruik eerst een mutex om de peripheral op te eisen...!
535         printf("AD-value: %d.\r\n", adValue);
536
537         // USART2 weer vrijgeven voor gebruik door een andere thread.
538         osMutexRelease(usart2TxMutexHandle);
539
540         // Even wachten.
541         osDelay(250);
542     }
543     /* USER CODE END AdConversion */
544 }
```



# Mutex

```
490 void Usart2Rx(void *argument)
491 {
492     /* USER CODE BEGIN Usart2Rx */
493     uint8_t receivedData = 0;
494
495     /* Infinite loop */
496     for(;;)
497     {
498         // Probeer één byte te ontvangen via USART2. Doe dat via 'polling' en wacht oneindig lang...
499         // Zonder het concept threads (tasks) in een RTOS, zou deze code blokkeren tot er een byte
500         // ontvangen wordt. Nu zal deze thread de voorziene tijd krijgen van de scheduler, daarna wordt
501         // overgestapt naar een andere thread (indien nodig). Deze thread krijgt daarna opnieuw de aandacht
502         // van de CPU om te kijken of er één byte is aangekomen. Deze afwisseling van threads herhaalt zich
503         // continu.
504         HAL_UART_Receive(&huart2, (uint8_t*)&receivedData, 1, HAL_MAX_DELAY);
505
506         // Als er een byte ontvangen is, toon hem op de LED's.
507         ByteToLeds(receivedData);
508     }
509     /* USER CODE END Usart2Rx */
510 }
```

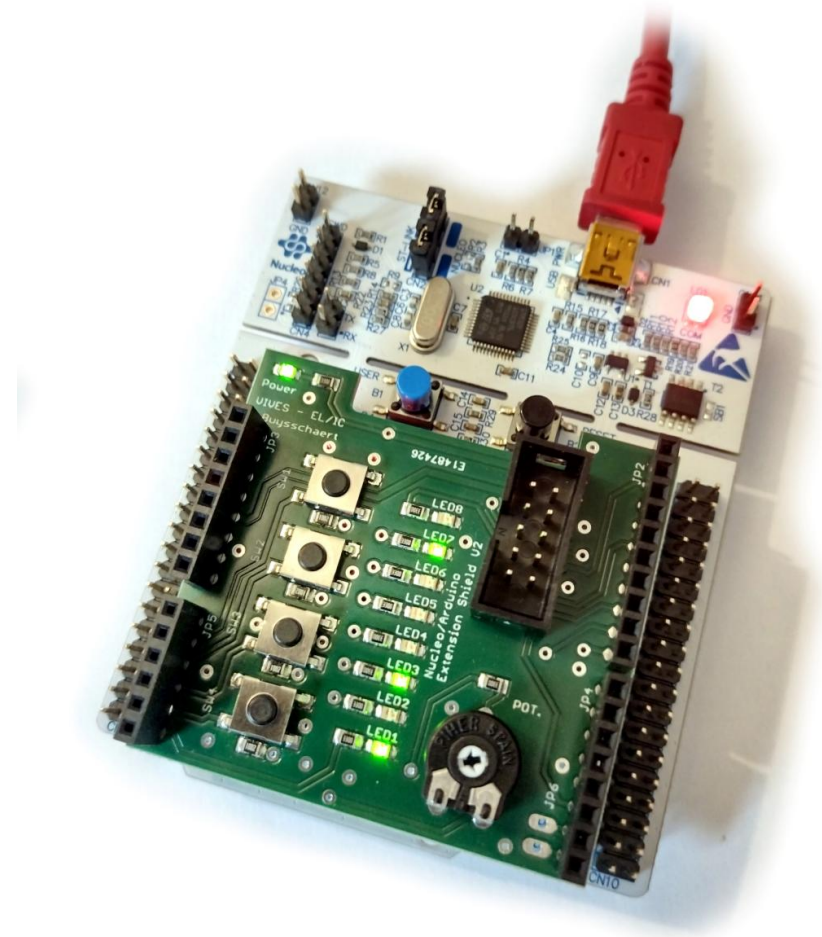


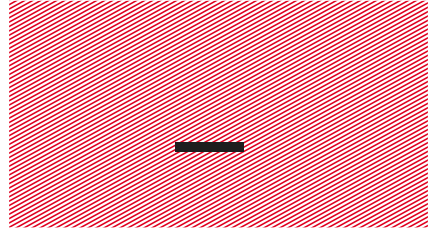
# Mutex

```
436 void BlinkLed1(void *argument)
437 {
438     /* USER CODE BEGIN 5 */
439     /* Infinite loop */
440     for(;;)
441     {
442         // User LED toggle'n.
443         HAL_GPIO_TogglePin(UserLED_GPIO_Port, UserLED_Pin);
444
445         // Even wachten (in Blocked status).
446         osDelay(500);
447     }
448     /* USER CODE END 5 */
449 }
```

# Mutex

```
COM22 - PuTTY
AD-value: 1354.
AD-value: 1408.
Hallo CMSIS RTOS V2...
AD-value: 1507.
AD-value: 1559.
AD-value: 1637.
Hallo CMSIS RTOS V2...
AD-value: 1682.
AD-value: 1751.
AD-value: 1761.
AD-value: 1762.
Hallo CMSIS RTOS V2...
AD-value: 1761.
AD-value: 1762.
AD-value: 1807.
AD-value: 1960.
Hallo CMSIS RTOS V2...
AD-value: 2028.
AD-value: 1629.
AD-value: 1621.
Hallo CMSIS RTOS V2...
AD-value: 1621.
AD-value: 1621.
A
```



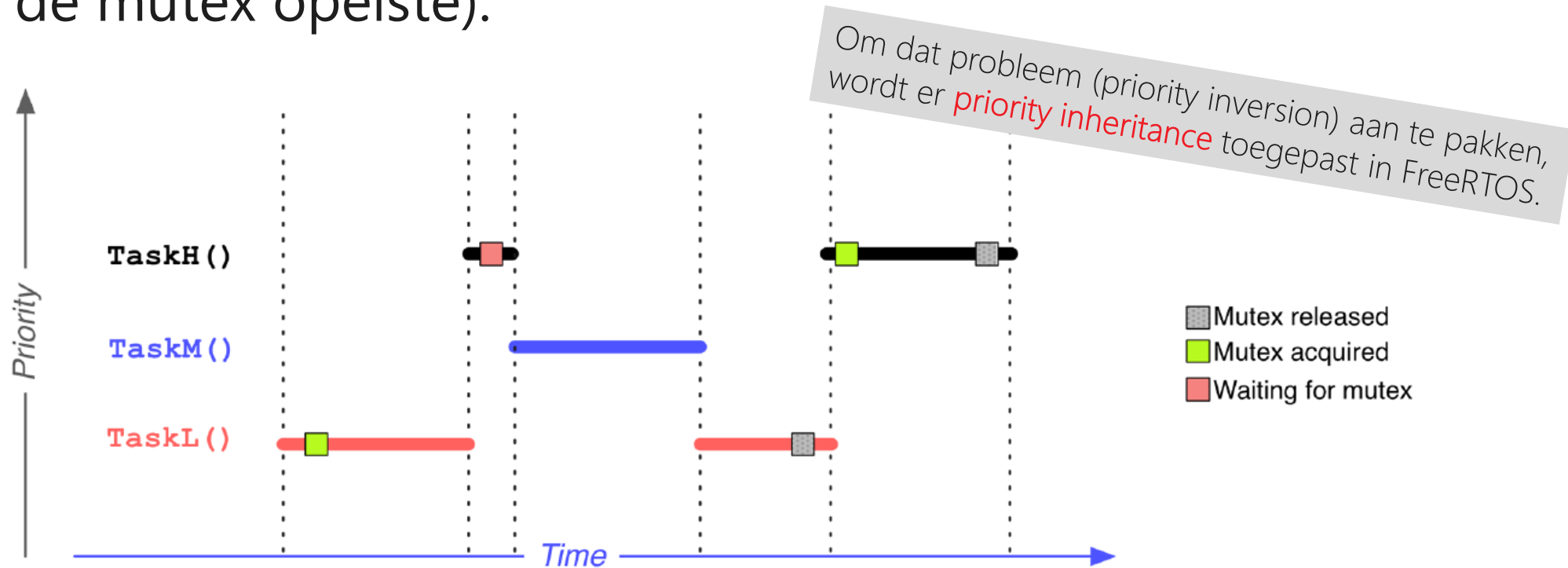


# Priority inversion



## Priority inversion

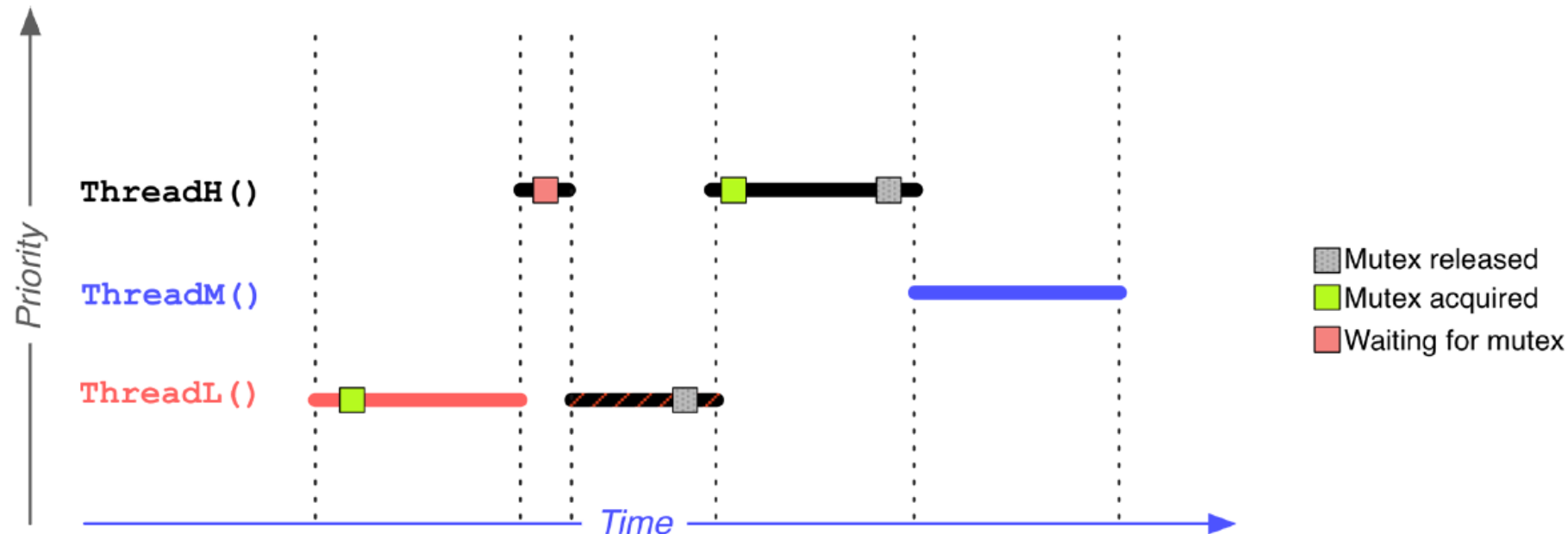
Soms kan het zijn dat er door het gebruik van 'mutexes' een **prioriteitsprobleem** ontstaat. Hieronder zie je dat de taak met **hogere prioriteit moet wachten** op een thread met lagere prioriteit (die de mutex opeiste).

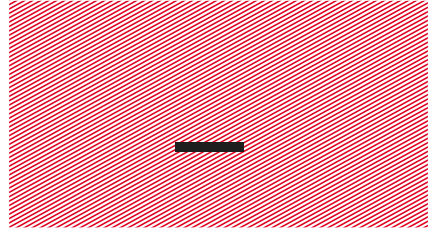




## Priority inversion

Als het RTOS vaststelt dat er priority inversion is, wordt de **prioriteit** van de thread die de mutex opeiste, **tijdelijk verhoogd** naar de hoogst nodige prioriteit... *ThreadM()* wordt hierdoor dus verschoven naar achter.





# Software Timers





- **Software timer** = een softwarematig aangemaakte timer die een **callback functie** oproept na een bepaalde ingestelde tijd.
- Te vergelijken met een hardware timer, **maar** niet even nauwkeurig!
- De timing gebeurt aan de hand van de RTOS kernel.
- Kan ingesteld staan als:
  - **one shot** (timer once).
  - continu (timer periodic).

# Software Timers

STM32CubeMX ondersteunt het maken van 'software timers'...

The screenshot shows the STM32CubeMX interface for a Nucleo-L476RG board. The 'Pinout & Configuration' tab is active, and the 'FREERTOS Mode and Configuration' section is expanded. The 'Timers and Semaphores' option is selected, and the 'Timers' table is visible. A red box highlights the 'softwareTimer' entry in the table.

| Timer Name    | Callback              | Type        | Code Generation Option | Parameter | Allocation | Control Block Name |
|---------------|-----------------------|-------------|------------------------|-----------|------------|--------------------|
| softwareTimer | SoftwareTimerCallback | osTimerOnce | Default                | NULL      | Dynamic    | NULL               |

Below the table, there are sections for 'Binary Semaphores' and 'Counting Semaphores', each with an 'Add' button.

Het aanmaken van een **nieuwe software timer** kan in C-taal met de functie `osTimerNew()`.

```
osTimerId_t osTimerNew ( osTimerFunc_t    func,  
                        osTimerType_t    type,  
                        void *           argument,  
                        const osTimerAttr_t * attr  
                        )
```

## Parameters

- [in] **func** function pointer to callback function.
- [in] **type** `osTimerOnce` for one-shot or `osTimerPeriodic` for periodic behavior.
- [in] **argument** argument to the timer callback function.
- [in] **attr** timer attributes; NULL: default values.

## Returns

timer ID for reference by other functions or NULL in case of error.

The function **osTimerNew** creates an one-shot or periodic timer and associates it with a callback function with *argument*. The timer is in stopped state until it is started with **osTimerStart**. The function can be safely called before the RTOS is started (call to **osKernelStart**), but not before it is initialized (call to **osKernelInitialize**).

The function **osTimerNew** returns the pointer to the timer object identifier or **NULL** in case of an error.

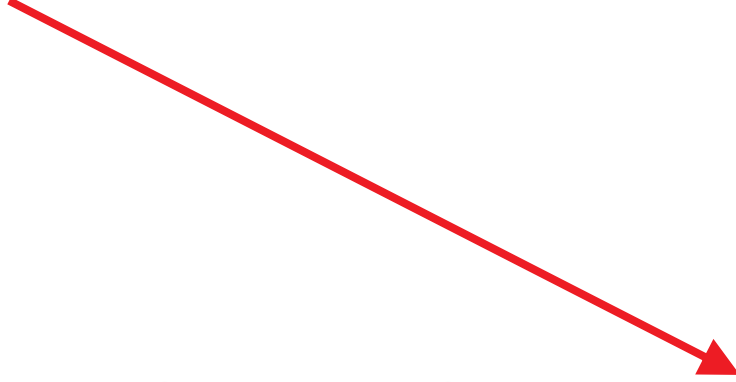
Bron: [https://arm-software.github.io/CMSIS\\_6/latest/RTOS2/group\\_CMSIS\\_RTOS\\_TimerMgmt.html#gad4e7f785c5f700a509f55a3bf6a62bec](https://arm-software.github.io/CMSIS_6/latest/RTOS2/group_CMSIS_RTOS_TimerMgmt.html#gad4e7f785c5f700a509f55a3bf6a62bec)

## Software Timers

Het aanmaken van een **nieuwe software timer** kan in C-taal met de functie `osTimerNew()`.

```
59  /* Definitions for softwareTimer */
60  osTimerId_t softwareTimerHandle;
61  const osTimerAttr_t softwareTimer_attributes = {
62      .name = "softwareTimer"
63  };
```

```
140  /* creation of softwareTimer */
141  softwareTimerHandle = osTimerNew(SoftwareTimerCallback, osTimerOnce, NULL, &softwareTimer_attributes);
```



# Software Timer

```
52  /* Definitions for buttonReadThrea */
53  osThreadId_t buttonReadThreaHandle;
54  const osThreadAttr_t buttonReadThrea_attributes = {
55      .name = "buttonReadThrea",
56      .stack_size = 128 * 4,
57      .priority = (osPriority_t) osPriorityLow,
58  };
```

```
64  /* Definitions for usart2Mutex */
65  osMutexId_t usart2MutexHandle;
66  const osMutexAttr_t usart2Mutex_attributes = {
67      .name = "usart2Mutex"
68  };
```

```
152  /* creation of buttonReadThrea */
153  buttonReadThreaHandle = osThreadNew(ButtonRead, NULL, &buttonReadThrea_attributes);
```

```
128  /* creation of usart2Mutex */
129  usart2MutexHandle = osMutexNew(&usart2Mutex_attributes);
```

# Software Timer

```
414 void ButtonRead(void *argument)
415 {
416     /* USER CODE BEGIN 5 */
417     bool swlHelp = false;
418     osStatus_t status;
419
420     /* Infinite loop */
421     for(;;)
422     {
423         // LED's inschakelen en software timer starten bij dalende flank.
424         // OPM: een software timer starten vanuit een ISR geeft een error (osErrorISR).
425         if(SWlActive() && (swlHelp == false))
426         {
427             swlHelp = true;
428
429             // Alle LED's inschakelen.
430             ByteToLeds(255);
431
432             // Timer starten die 3 seconden loopt (one shot).
433             status = osTimerStart(softwareTimerHandle, 3000);
434
435             if(status == osOK)
436             {
437                 // Melding geven (maar eest mutex opeisen).
438                 // OPM: dit zou ook kunnen met een binaire semafoor, maar dan heb je geen 'priority inheritance'
439                 // voor mocht het nodig zijn (kans op dead locks).
440                 osMutexAcquire(usart2MutexHandle, osWaitForever);
441                 printf("Software timer started.\r\n");
442                 osMutexRelease(usart2MutexHandle);
443             }
444         }
445     }
```

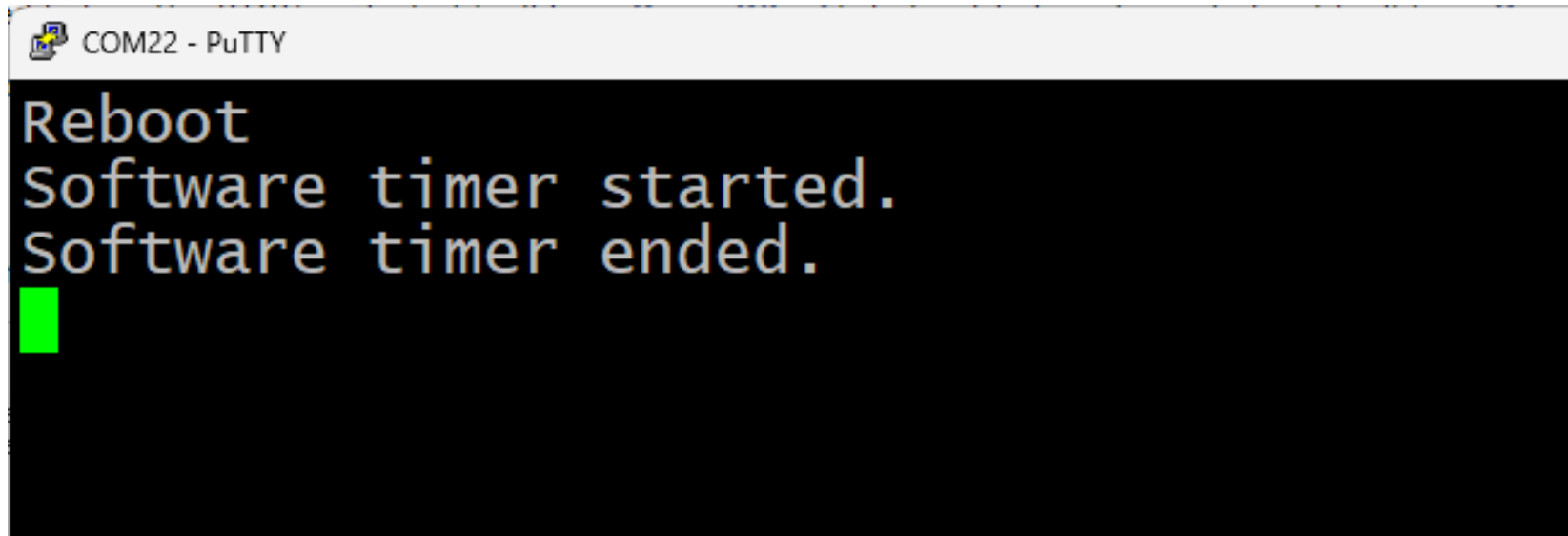
# Software Timer

```
455 /* SoftwareTimerCallback function */
456 void SoftwareTimerCallback(void *argument)
457 {
458     /* USER CODE BEGIN SoftwareTimerCallback */
459     // LED's terug uitschakelen (na 3 seconden).
460     ByteToLeds(0);
461
462     // Melding geven (maar eerst mutex opeisen).
463     osMutexAcquire(usart2MutexHandle, osWaitForever);
464     printf("Software timer ended.\r\n");
465     osMutexRelease(usart2MutexHandle);
466     /* USER CODE END SoftwareTimerCallback */
467 }
```

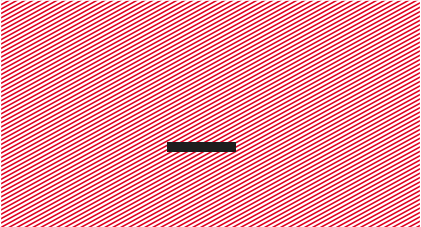


—

# Software Timer



```
COM22 - PuTTY
Reboot
Software timer started.
Software timer ended.
█
```



# ESP32 C3 en CMSIS RTOS V2

# ESP32 C3 en CMSIS RTOS V2

## Doel?

Een IoT toegangspoort te maken naar je vakantievilla ;-).



## Hoe?

De WiFi-module met ESP32 C3 chip (LILYGO T-01C3) combineren met CMSIS RTOS V2... En de informatie visualiseren op een website.

Om alles in kleine modules te kunnen programmeren, maken we (veelvuldig) gebruik van threads:

- **watchdogTask** = reset een watchdog timer iedere seconde.
- **esp32Task** = neemt de communicatie via AT-commando's met de ESP32-module voor zijn rekening.
- **adConvTask** = meet de AD-waarde op van de on board trimmer.
- **stateMachineTask** = houdt bij in welke state de poort staat en wat eventueel de volgende state moet zijn.
- **switchesTask** = leest de on board drukknoppen in.

Dit project gebruikt naast threads ook nog vele andere RTOS concepten, namelijk:

- drie **queues**.
- twee **timers**.
- één **mutex**.
- twee **event flags**.
- ...

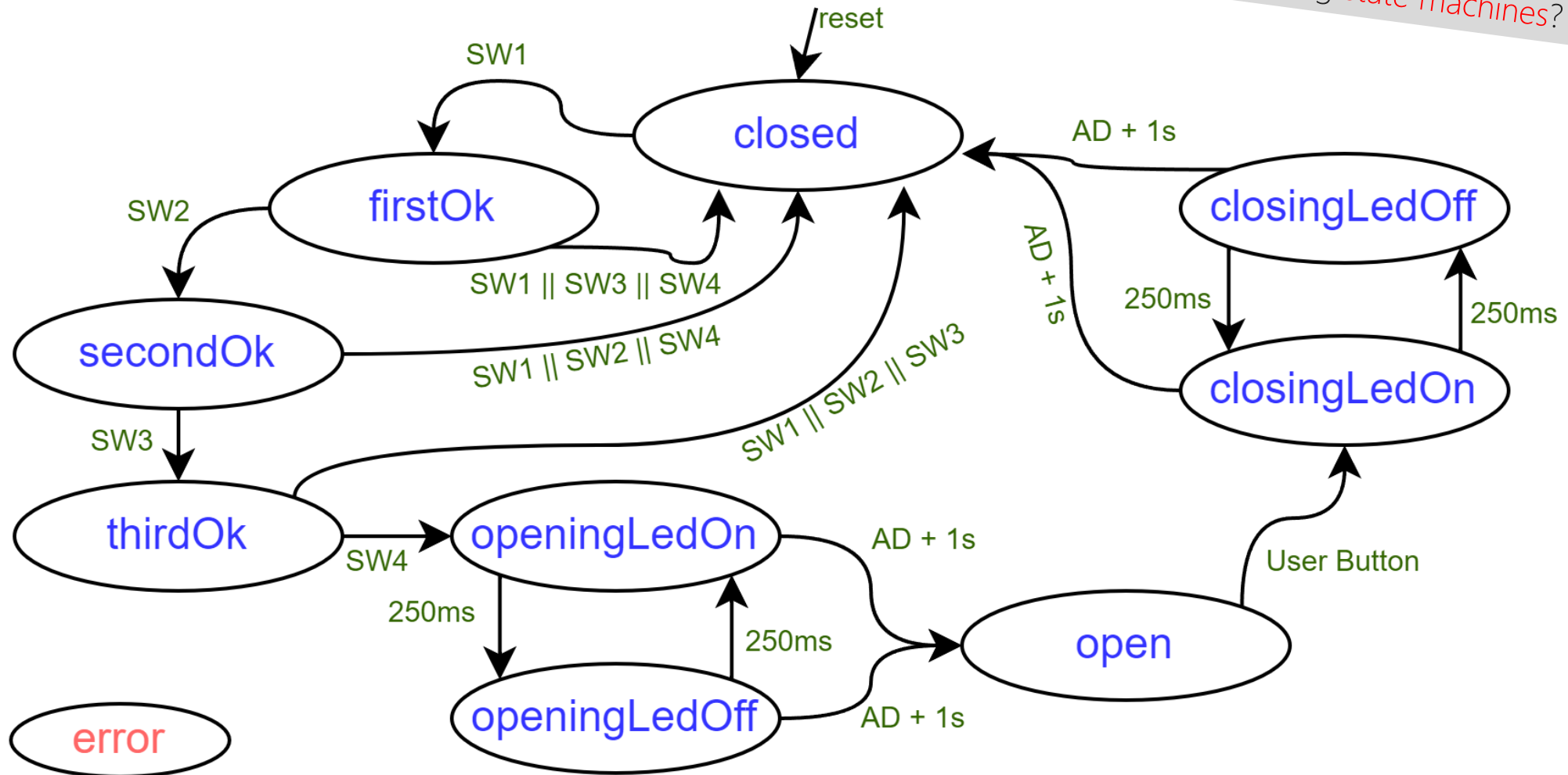
Als motor gebruiken we een modelbouwservo.

Open het STM32CubeMX project en **zoek** al die items eens op.



# ESP32 C3 en CMSIS RTOS V2

Ken je nog *state machines*?



```
145 enum States {error, closed, firstOk, secondOk, thirdOk, openingLedOn, openingLedOff,  
146             open, closingLedOn, closingLedOff};
```



# ESP32 C3 en CMSIS RTOS V2

Sommige threads hebben een **grotere stack size** nodig om vlot te kunnen werken. Dat komt vooral omdat die threads met vrij veel seriële data moeten werken.

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc: STM32L476RGTx NUCLEO-L476RG

STM32CubeMX

FileWindowHelpmyST

HomeSTM32L476RGTx - NUCLEO-L476RGCubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration

Pinout & ConfigurationClock ConfigurationProject Manager

Software PacksPinout

FREERTOS Mode and Configuration

InterfaceCMSIS\_V2

Configuration

Reset Configuration

User ConstantsTasks and QueuesTimers and SemaphoresMutexesEventsFreeRTOS Heap Usage

Config parametersInclude parametersAdvanced settings

Tasks

| Task Name       | Priority              | Stack Size (Words) | Entry Function | Code Generation Opt... | Parameter | Allocation | Buffer Name | Control Block Name |
|-----------------|-----------------------|--------------------|----------------|------------------------|-----------|------------|-------------|--------------------|
| watchdogTask    | osPriorityBelowNormal | 128                | Watchdog       | Default                | NULL      | Dynamic    | NULL        | NULL               |
| esp32Task       | osPriorityNormal      | 1280               | Esp32          | Default                | NULL      | Dynamic    | NULL        | NULL               |
| adConvTask      | osPriorityNormal      | 128                | AdConversion   | Default                | NULL      | Dynamic    | NULL        | NULL               |
| stateMachineTas | osPriorityNormal      | 1280               | StateMachine   | Default                | NULL      | Dynamic    | NULL        | NULL               |
| switchesTask    | osPriorityNormal      | 128                | Switches       | Default                | NULL      | Dynamic    | NULL        | NULL               |

AddDelete

Queues

| Queue Name  | Queue Size | Item Size | Allocation | Buffer Name | Control Block Name |
|-------------|------------|-----------|------------|-------------|--------------------|
| adQueue     | 1          | uint8_t   | Dynamic    | NULL        | NULL               |
| switchQueue | 5          | int8_t    | Dynamic    | NULL        | NULL               |
| mqttQueue   | 3          | uint8_t   | Dynamic    | NULL        | NULL               |

AddDelete

Middleware and Software Pa...

AIROC-Wi-Fi-Bluetooth-ST...

FATFS

FP-ATR-SIGFOX1

FP-SNS-FLIGHT1

FP-SNS-MOTENV1

FP-SNS-SMARTAG2

FREERTOS

I-CUBE-Cesium

I-CUBE-ITTIADB

I-CUBE-embOS

I-CUBE-wolfSSL

I-Cube-SoM-uGOAL

TOUCHSENSING

USB\_DEVICE

USB\_HOST

X-CUBE-AI

X-CUBE-ALGOBUILD

X-CUBE-ALS

X-CUBE-AZRTOS-L4

X-CUBE-BLE1

X-CUBE-BLE2

X-CUBE-BLEMGR

X-CUBE-DISPLAY

X-CUBE-EPRMA1

X-CUBE-GNSS1

X-CUBE-ISPU

X-CUBE-MEMS1

X-CUBE-NFC4

X-CUBE-NFC6

# ESP32 C3 en CMSIS RTOS V2

Refresh de watchdog timer in een thread met lage prioriteit. Op die manier zorg je ervoor dat de microcontroller reset als één van de threads met hogere prioriteit vast zit.

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc: STM32L476RGtx NUCLEO-L476RG

STM32CubeMX File Window Help

Home STM32L476RGtx - NUCLEO-L476RG CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration

Pinout & Configuration Clock Configuration Project Manager

Software Packs Pinout

IWDG Mode and Configuration

Mode

Activated

Configuration

Reset Configuration

Parameter Settings User Constants

Configure the below parameters :

Search (Ctrl+F)

Watchdog Clocking

IWDG counter clock prescaler 256

IWDG window value 4095

IWDG down-counter reload value 999

$32\text{kHz}/256 \times 1000 = 8 \text{ seconden...}$

# ESP32 C3 en CMSIS RTOS V2

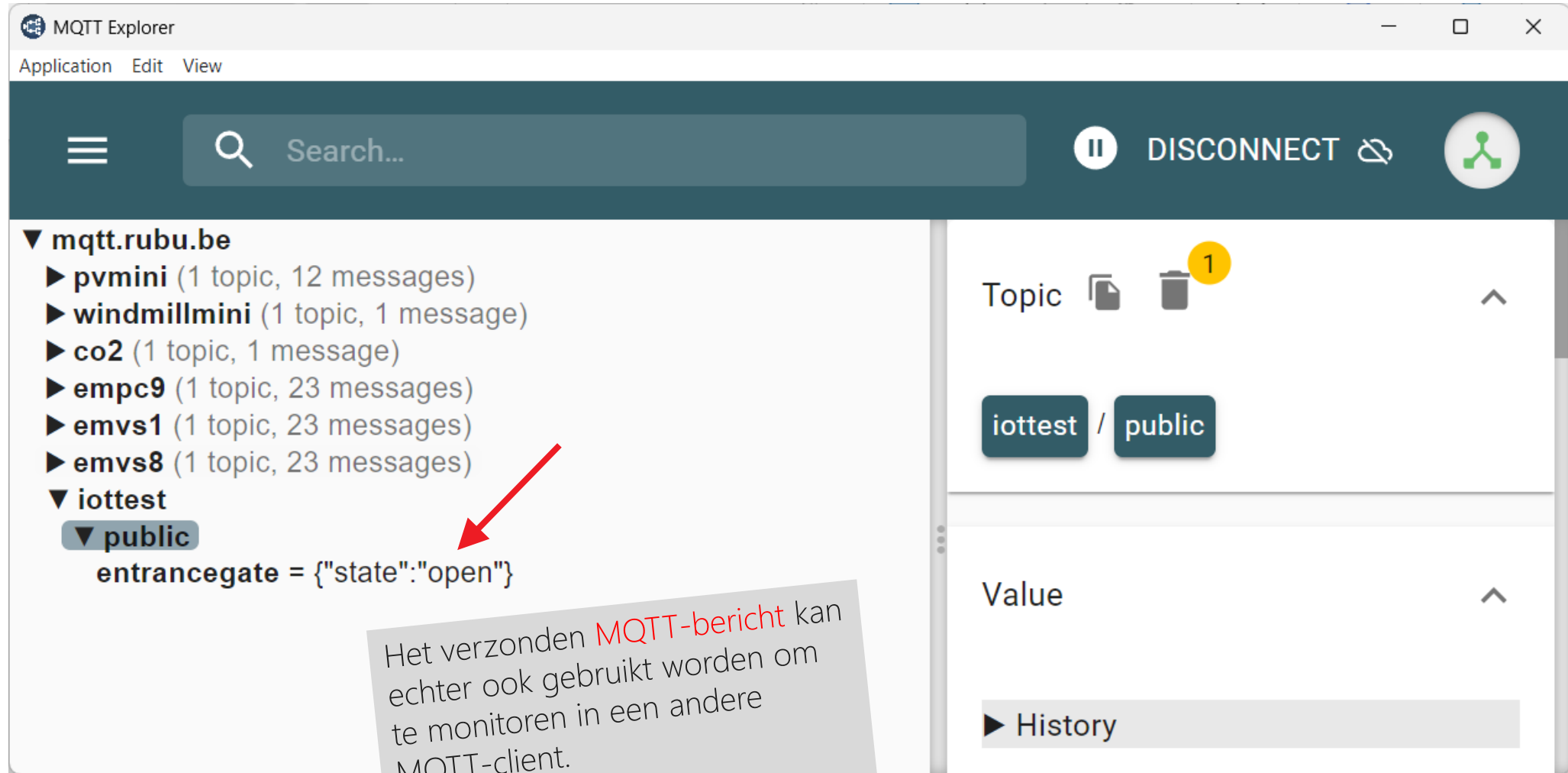
Verzend een **MQTT-bericht** als de poort geopend of gesloten werd.

```
809  /* Infinite loop */
810  for(;;)
811  {
812      // Is er een nieuwe state opgehaald?
813      osStatus = osMessageQueueGet(mqttQueueHandle, &state, NULL, 1);
814      if(osStatus == osOK)
815      {
816          if(state == open)
817          {
818              // MQTT-bericht publish'n.
819              MqttPublish(MQTT_PUB_TOPIC, "{\"state\":\"open\"}");
820          }
821
822          if(state == closed)
823          {
824              // MQTT-bericht publish'n.
825              MqttPublish(MQTT_PUB_TOPIC, "{\"state\":\"closed\"}");
826          }
827      }
828
829      // Mogelijks ontvangen TCP-, HTTP-, HTTPS-, of MQTT-berichten zoeken in het circulair buffer.
830      // De berichten zijn van de vorm: +IPD, of +MQTTSUBRECV:0,"rubu/esp8266",3,254.
831      while(circularBuffer.numberOfStringsInBuffer > 0)
832      {
833          // Eén string ophalen.
834          circularBufferActionResult = PopStringFromCircularBuffer(&circularBuffer, receivedText);
835          if(circularBufferActionResult == readStringSucceeded)
836          {
```

# ESP32 C3 en CMSIS RTOS V2

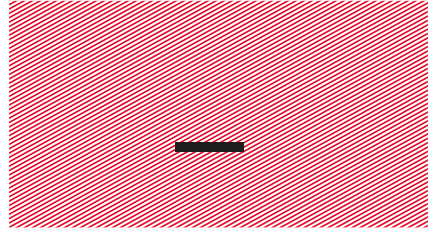


# ESP32 C3 en CMSIS RTOS V2



Het verzonden MQTT-bericht kan echter ook gebruikt worden om te monitoren in een andere MQTT-client.

Bijvoorbeeld: MQTT Explorer.



# Theorieopdracht





## Theorieopdracht

---

Maak gebruik van de demo code i.v.m. het gebruik van de WiFi-module en de toegangspoort om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - CMSIS RTOS V2 - WiFi ESP32 C3 - toegangspoort - template'.

Zie: **"Theorieopdracht 8 – CMSIS RTOS V2 - Toegangspoort"**.